

The 3 Pillars of Robotics

A practical roadmap for getting started

Robotics is not one skill. It is the intersection of three engineering domains: software, electrical, and mechanical. Most beginners overfocus on one and ignore the others. That is why so many projects never fully work.

If you want to build real robots, you need working knowledge in all three.

1. Software

The brain of the robot

Software decides how the robot behaves. It processes sensor data, makes decisions, and controls motion. But robotics software is not just writing Python scripts. It is about understanding how code interacts with hardware and time.

Start by building strong fundamentals in programming. You should understand memory, data structures, and how to debug properly. After that, move into C or C++. These languages force you to think closer to the hardware, which is essential in robotics.

Control systems are non-negotiable. Learn what a feedback loop is. Implement a PID controller from scratch. Understand how a system behaves when it is unstable, noisy, or delayed.

You should also get comfortable reading sensor data and dealing with noise. Real world signals are never clean. Learn simple filtering techniques and understand why they matter.

To get started, program a microcontroller. Read a sensor. Control a motor. Then build something autonomous, even if it is small, such as a line follower or a balancing robot. That is where theory becomes real.

When it comes to learning coding, especially if you know nothing, do the following:

Do some tutorial videos, for 1-3 hours. Then find a simple project and do that. Especially when we now have tools like Claude Code and ChatGPT. You have all the help in the world to get started and learn it completely.

Python:

<https://www.youtube.com/watch?v=K5KVEU3aaeQ>

<https://www.youtube.com/watch?v=rfscVS0vtbw>

C++:

<https://www.youtube.com/watch?v=-TkoO8Z07hl>

C:

<https://www.youtube.com/watch?v=xND0t1pr3KY>

2. Electrical

The heart of the robot

Electrical engineering is where many robotics projects fail. Not because it is impossible, but because it could be extremely difficult. Something could break, and you'll never know why.

You need to understand how power flows through your system. Voltage, current, resistance, and power are not optional concepts. They determine whether your robot runs smoothly or randomly resets.

Learn how to select batteries properly. Understand voltage regulation. Know how to distribute power safely. Motors draw large currents. Sensors are sensitive to noise. If you do not respect this, your system will behave unpredictably.

You also need to understand motor drivers and signal types. What is the difference between analog and digital signals. How does an ADC work. Why do we use capacitors for filtering. These are foundational ideas that make your system stable.

Start simple. Power a motor from a battery using a driver. Read an analog sensor correctly. Practice measuring voltage and current with a multimeter. Learn to debug methodically instead of guessing.

Where i would recommend to start is here:

Buy a Arduino starter set, and do some tutorials for each sensor:

<https://www.aliexpress.com/w/wholesale-arduino-starter-kit.html>

After this, you need to find a project or use case for yourself, and let curiosity drive you forward.

To learn PCB design look at tools like KiCad and flux.ai. For how you make them i'd recommend follow 1-2 tutorials like these:

<https://www.youtube.com/watch?v=pNhH6LIU7Mw>

<https://www.youtube.com/watch?v=0cwvYz2HmLw>

<https://www.youtube.com/watch?v=TvA4ut0o4bc>

3. Mechanical

The body of the robot

Software cannot compensate for bad mechanics. If the structure is too heavy, poorly aligned, or mechanically inefficient, no amount of clever code will fix it.

You need to learn CAD. Not just how to draw parts, but how to design assemblies that actually fit together. Understand tolerances, clearances, and how real materials behave.

Robotics is physical. Motors have torque limits. Structures bend. Friction exists. You must understand basic statics, torque calculations, and load paths. Even simple calculations dramatically improve your designs.

Also learn how things are manufactured. 3D printing has constraints. Fasteners matter. Bearings matter. Small design decisions make large differences in performance.

To begin, design and print a simple bracket. Calculate the torque needed for a joint before choosing a motor. Build small mechanical subsystems and test them physically.

To get started with CAD I would recommend doing 6-7 of these videos, and then make something from your own imagination.

https://www.youtube.com/watch?v=d3qGQ2utl2A&list=PLrZ2zKOtC_-C4rWfapngngoe9o2-ng8ZBr&index=4

For this I use fusion360 or onshape, both that have free plans.

Bringing It All Together

The biggest mistake is learning these fields in isolation.

The fastest way to grow is to design a small robot and force yourself to handle everything. Design the structure. Wire the electronics. Program the control. Debug the failures.

That is where real learning happens.

Start small. But build complete systems.